Smart Travel Planner

An agent that matches travelers to destinations — agents, tools, RAG, and ML classification

The Project

A friend texts you: "I have two weeks off in July and around $1,500. I want somewhere warm, not too touristy, and I like hiking. Where should I go, when should I book, and what should I expect?"

Build the system that answers them. An agent that figures out what kind of trip the person actually wants, pulls up what it knows about destinations matching that vibe, checks what flights and weather actually look like right now, and delivers a real plan to a real channel. You will compose everything from the past three weeks into one system that could plausibly help someone book a trip tomorrow.

This week is the difficult one. The brief tells you what must be true; you decide how. If you find yourself waiting for a step-by-step, that is the assignment showing you where you need to grow.

Two things matter equally: that the AI works, and that the code around it is built like an engineer built it. We will read your code with both eyes open. Junior-level habits — globals, blocking I/O, copy-pasted clients in every function, magic strings — will cost you here.

What You Must Build

1. An ML Classifier

Train a model that classifies destinations by travel style: Adventure, Relaxation, Culture, Budget, Luxury, Family. You compile the dataset yourself — 100 to 200 destinations, labeled by you, with features you justify. Your README must explain your labeling rules and your features; "I picked them because they seemed right" is not justification.

- Use a proper scikit-learn Pipeline. Preprocessing lives inside the pipeline. We will check for leakage.
- Compare at least three classifiers with k-fold cross-validation. Report accuracy and macro F1 with mean and standard deviation.
- Tune at least one model. Show what you searched and why.
- Address class imbalance honestly — some travel styles will be rare. Report per-class metrics, not just averages.
- Track every experiment in a results.csv (model, params, metrics, timestamp). Pin your dependencies. Fix your seeds. Save the winner with joblib.

2. A RAG Tool

Build a retrieval system over real destination content (Wikivoyage, travel blogs, tourism boards). 10–15 destinations, 20–30 documents, embeddings stored in Postgres via pgvector — same database as the rest of your app. Justify your chunk size, your overlap, and your retrieval strategy in the README. Show that you tested your retrieval on a few hand-written queries before plugging it into the agent.

3. An Agent With Three Tools

Use LangGraph or LangChain. The agent has three tools: one for retrieving destination knowledge, one for classifying a destination's travel style using your trained model, and one for fetching live conditions (weather, flights, FX — pick your APIs).

- Every tool input is validated by a Pydantic schema before the function runs. If the LLM sends garbage, the agent retries. It does not crash.
- Maintain an explicit tool allowlist. Anything else is refused, even if the model invents it.
- Trace your agent end-to-end (LangSmith free tier or equivalent). Include a screenshot of a multi-tool trace in your README.

- The agent must genuinely synthesize across tools, not concatenate. If RAG and the live API tell different stories, the answer should reflect that tension.

4. Two Models, One Agent

Don't use your most expensive model for every step. Route a cheap one (Haiku-class, gpt-4o-mini) to mechanical work like extracting tool arguments and rewriting RAG queries; route a stronger one to the final synthesis. Log token usage per step. Report the cost of one full query in your README. Be honest with the number.

5. Persistence — Postgres + pgvector + SQLAlchemy

One database for everything: users, agent runs, tool-call logs, and embeddings. Decide what to persist and why. At minimum: who asked, what they asked, what the agent answered, which tools fired, and when. Use SQLAlchemy for relational models. Migrations via Alembic if you can manage it.

6. Auth — Sign-Up and Login

Real users, real accounts. Registration, login, password hashing, sessions or JWTs — your call. Every agent run is scoped to the logged-in user; history and webhook destinations belong to that user. You have a reference app from earlier in the bootcamp; use it for inspiration, not for copy-paste. One role is fine.

7. React Frontend

A real React app over your FastAPI backend. Sign-in flow, a chat-style interface for trip questions, and a way for the user to see what the agent did — which tools fired, what they returned. Vite + React is the fastest path. Stream the response if you can.

8. Webhook Delivery

When the agent finishes, fire a webhook delivering the trip plan to a real channel: Discord, Slack, a Sheets append via n8n/Make, email via Resend — your choice. Add a timeout, at least one retry with backoff, and structured logging on failure. The webhook failing must not break the user-facing response.

9. Docker — the Whole Stack

Containerize the backend, frontend, and Postgres. A docker-compose.yml that brings the whole thing up with one command. Use a named volume for Postgres so your embeddings and user data survive container restarts. If a reviewer can't run your project with `docker compose up`, you have not finished.

How You Build It — Engineering Standards

These apply across every feature above. Treat them as part of the spec, not as nice-to-haves.

Async All the Way Down

Your FastAPI routes are async. Your tool functions are async. Your database calls are async (SQLAlchemy 2.x async session, or asyncpg). Your HTTP calls go through httpx.AsyncClient, not requests. Your LLM SDK calls use the async methods. If you have a single `time.sleep` or `requests.get` in a request path, you have blocked the event loop — find it and fix it. Agents are I/O-bound systems; doing this synchronously is incorrect, not just slow.

Dependency Injection — Use FastAPI's Depends

Stop instantiating clients inside route handlers. Your LLM client, database session, embedding model, agent executor, and current user are all dependencies — declare them with `Depends()` and let FastAPI wire them in. This is how the framework was designed to work. It also makes your code testable: in tests you override the dependency, you don't monkey-patch globals.

Singletons — Done Right

Some things should exist exactly once per process: the database engine, the loaded ML model, the vector store connection, the embedding model, the LLM client. Manage them through FastAPI's lifespan handler — create on startup, dispose on shutdown, expose through dependencies. Loading the joblib model on every request is a bug, not a style choice. Globals scattered across modules is also a bug; the lifespan + DI pattern is the right answer.

Caching — Use lru_cache and TTL Caches Where They Pay Off

`functools.lru_cache` on functions that are deterministic and expensive: settings loaders, model file paths, anything that reads config. A TTL cache (cachetools or aiocache) on tool responses where it makes sense — weather for the same city within 10 minutes is the same answer; don't pay the API twice. Document where you cached and why. Caching the wrong thing is worse than not caching.

Configuration — pydantic-settings, Not Magic Strings

All configuration goes through a single Settings class built on pydantic-settings. Environment variables are typed and validated at startup; if a required key is missing, the app refuses to start — it does not fail mysteriously on the third request. No `os.getenv` scattered through your code. No string literals for model names, API URLs, or queue names buried in modules. One source of truth.

Type Hints, Pydantic, and the Boundary

Every function in your codebase has type hints. Every external boundary — HTTP request bodies, tool inputs, LLM structured outputs, webhook payloads — is a Pydantic model. Pydantic is your fence: data is validated when it crosses in, and after that you trust your types. Don't validate the same thing five times in five places; validate at the edge.

Errors, Retries, and Failure Isolation

Every external call (LLM, tool API, webhook) can and will fail. Wrap them with timeouts and retries with backoff (tenacity is fine; a hand-rolled async retry decorator is fine too). When a retry budget is exhausted, the failure is logged with structure and the agent recovers gracefully — never let a transient flight-API outage take down the whole request. Tool failures inside the agent loop should be returned to the LLM as structured errors so it can reason about them, not raised as Python exceptions that crash the run.

Code Hygiene

A real project layout: routes, services, models, tools, and agent code in their own modules — not one 600-line main.py. Logging with structlog or the stdlib's logger configured for JSON — no print statements. Linters and formatters in the repo (ruff, black, or equivalent) and a pre-commit config that runs them. A .env.example listing every required key. A README that explains the architecture, not just how to run it.

Tests — At Least the Critical Path

You don't need 100% coverage. You do need: a test that exercises each tool in isolation with a fake LLM, a test of your Pydantic schemas with both valid and invalid inputs, and one end-to-end test of the agent with mocked external APIs. Tests run in CI (GitHub Actions) on every push. If your tests don't run automatically, they will rot.

OPTIONAL — GO FURTHER

Pick what interests you. Don't start any of these until your required nine work end to end.

MLflow or Weights & Biases OPTIONAL

Replace your results.csv with proper experiment tracking. Log every run, every artifact, every metric. Screenshot the dashboard for your README.

Structured Logging — SEQ or Similar OPTIONAL

Send structured logs from FastAPI to SEQ, Grafana Loki, or Better Stack. Log every agent run, every tool call, every failure. The point is to be able to reconstruct exactly what happened from the logs alone.

Secrets Management OPTIONAL

Move keys out of .env into HashiCorp Vault, Doppler, or Infisical. Overkill for homework, valuable to learn once. At minimum, structure your config with pydantic-settings.

Deploy It OPTIONAL

Ship it somewhere real. Backend on Railway, Fly.io, or Render. Database on Supabase (pgvector supported) or Neon. Frontend on Vercel. Send the URL to a friend and have them plan a real trip.

Push the Agent Itself OPTIONAL

Human-in-the-loop approval before the webhook fires. A caching layer for the live APIs so repeated queries don't burn your free tier. A "compare two destinations" mode. A planner-then-executor agent vs. ReAct comparison with a written reflection. Surprise us.

Deliverables

One GitHub repo containing everything above plus a README with: an architecture diagram you drew, your dataset labeling rules, your chunking and retrieval rationale, your model comparison table, a per-query cost breakdown, a LangSmith trace screenshot, and a list of any optional extensions you completed. Plus a 3-minute demo video showing one end-to-end run from the React UI through to the webhook firing.

How You'll Be Evaluated

On four things: whether the system actually works end to end; whether the AI parts are done with care (no leakage, real validation, real synthesis, traceable runs); whether the engineering is real (async, DI, lifespan singletons, typed boundaries, no globals, no blocking I/O); and whether you can defend your choices. Expect to be asked why you cached that response, why that's a Depends and not a global, why you chunked the way you did, why those three classifiers. "The tutorial said so" is the wrong answer. "I tried X, it failed because Y, so I did Z" is the right one.

Build small. Trace everything. Defend every choice. — Hasan